Where It Lives Is Not What It Is: An Architectural Vocabulary for Retained Adaptation in Agentic Systems

Zbigniew Łukasiak

Independent Researcher
research@lukasiak.me

Abstract. Agentic systems increasingly adapt by retaining behavior-shaping artifacts such as prompts, workflows, validators, routing rules, memories, adapters, and checkpoints. These artifacts are managed and often classified by storage substrate, such as memory systems, flat files, prompt registries, repositories, vector stores, databases, or model-artifact stores. This position paper argues that storage-first classification is architecturally insufficient. Retained behavior-shaping artifacts should instead be classified by the characteristics that determine how they affect behavior: how their operative parts are represented and consumed, where they came from, and what behavioral authority they have.

The vocabulary records four properties of retained behavior-shaping artifacts: storage substrate, representational form, lineage, and behavioral authority. Storage remains important, but only as one field among others. Representational form distinguishes prose such as prompts, symbolic artifacts such as code, and distributed-parametric artifacts such as embeddings, adapters, and weights: the same retained lesson may be cheap to capture as prompt text but cheaper to execute and easier to test as symbolic control. Each form has its own default inspection method: prose can be read directly, symbolic artifacts can be tested or checked, and distributed-parametric artifacts usually require behavioral probes. When objects bundle several behavior-shaping parts, the vocabulary applies to the relevant operative part or consumption path.

We frame this as an architectural view over retained behavior-shaping artifacts. The view supports architectural review of efficiency, security, and sovereignty, and helps architects choose the appropriate representational form for retained behavior. We apply the record to a corpus of 141 agent-memory systems reviewed from source code, where storage substrate discriminates little among artifacts that the other fields separate sharply.

Keywords: agentic systems · LLM agents · persistent adaptation · agent memory · software architecture · context engineering

1 Introduction

Agentic systems do not adapt only by changing model weights. Their behavior is also shaped by prompts, repositories, tools, validators, configuration, routing

systems, traces, and learned controllers. These retained artifacts shape future behavior in distinct ways, yet they are often discussed under broad labels such as memory, tool use, or prompt management.

Consider a coding agent that repeatedly mishandles dependency resolution: it installs incompatible versions, ignores lockfile constraints, or misses conflicts that appear only during tests. The surrounding system may retain the lesson as a prose reflection, project workflow, conflict-checking script, lockfile validator, derived prompt summary, or learned scoring weights.

These may all be lumped under broad labels, but they create different affordances and obligations for execution, review, invalidation, and rollback: a reflection affects behavior only if retrieval carries it forward; a validator blocks execution cheaply but must be tested like control logic; a stale derived artifact can govern behavior after its source is fixed; learned scoring weights require behavioral probes rather than reading one record.

The architectural question is therefore not only where retained state lives, but which retained form gives the system acceptable tradeoffs in efficiency, security, and sovereignty. Retained behavior-shaping artifacts can be modeled as architectural elements, or as retained state that defines, configures, connects, or constrains such elements; their properties matter because they determine how retained behavior is reviewed, enforced, and governed [14]. This paper captures those properties in four fields:

Field	Examples	Why the architect cares
Storage substrate	Reflection in vector store or flat file; validator in repository; prompt summary in registry.	Determines access, deletion, versioning, and rollback paths.
Representational form	Prose reminder; prose prompt content; symbolic executable validator; dense-vector retrieval index.	Determines default inspection methods (reading, tests, or probes), review evidence, and recurring execution cost.
Lineage	Prompt summary as derived view of canonical workflow.	Determines what must be invalidated when the source changes.
Behavioral authority	Reflection as advice; prompt as instruction; validator as enforcement.	Records who consumes the artifact, through which channel, and with what force.

This paper makes three claims:

– Retained adaptation should be recorded at the level of behavior-shaping operative parts and consumption paths, not merely stored objects.

- Four architectural record fields–storage substrate, representational form, lineage, and behavioral authority–capture affordances and obligations that storage labels hide, and guide the assignment of review evidence, invalidation triggers, and rollback paths.
- The resulting record supports architectural design and review decisions about whether retained behavior should remain prose, become symbolic control, be exposed through retrieval, or be learned into numerical state.

These fields expose recurring architectural risks: efficiency risk when retained lessons stay in recurring prompt or retrieval paths that cheaper symbolic checks, schemas, tests, or workflows could replace; security risk when untrusted or poisoned artifacts enter high-authority channels; and sovereignty risk when behavior depends on artifacts the owner cannot inspect, regenerate, delete, or roll back, such as vendor checkpoints or hosted retrieval indexes [2,3].

2 Retained Adaptation as an Architectural Concern

A retained behavior-shaping artifact is retained state that a later agentic loop can consume in a way that affects behavior. It enters the scope whenever it has been produced, selected, revised, or maintained in response to experience, supervision, evaluation, correction, or incident review. Below, **retained artifact** refers to this class unless otherwise specified. The inclusion boundary is behavioral consequence rather than implementation label.

Its **operative part** is the content, structure, parameterization, or behavior-affecting mechanism that a later model, runtime, router, validator, tool, or learning loop can consume or apply in a behavior-shaping way. Within this paper's vocabulary, operative parts may be prose, symbolic, or distributed-parametric.

A stored export or backup outside the system's consumption paths does not count merely because it is persisted. The same artifact counts once a later agentic loop can read, retrieve, execute, or learn from it in a behavior-shaping way; adaptation often moves into the surrounding architecture rather than only into explicit memory stores.

3 Relation to Existing Classifications

Architecture work on ML-based systems already treats models, data, and AI-related design decisions as architectural concerns [12,5]. Agent-specific work classifies mechanisms — memories, tools, skills, rules, workflows, context-management procedures, learned adaptation — and surveys how they are stored, retrieved, compressed, and updated [7,4,8,11,9,18,22,20].

These classifications are valuable for grouping artifacts by mechanism kind, storage, retrieval, compression, or update procedure. This paper records what those categories leave implicit: what evidence is needed to review a retained artifact, what source changes invalidate it, what channel gives it force, what rollback path removes its effect, and how form shifts recurring execution cost. A

mechanism label does not settle these — a "memory" can be low-authority prose advice, a derived high-priority prompt, a retrieval index, or learned scoring state, and the architectural obligations differ across those cases. Four record fields — substrate, form, lineage, authority — make those obligations explicit inside and across mechanism categories.

4 Storage Substrate: Where Retained State Lives

Storage substrate records where retained state persists: flat files, repositories, databases, vector stores, prompt registries, configuration services, audit logs, service objects, or model-artifact stores. It matters for access control, deletion, versioning, rollback, latency, and deployment.

However, the same substrate can host different operative forms and risks. A repository may contain prose workflows, symbolic validators, generated prompt views, route tables, and pointers to checkpoints. A vector store may package readable prose records together with dense-vector indexes and scoring procedures. Storage is therefore an operational field, not a sufficient representational or behavioral classification.

5 Representational Form

Representational form asks how an artifact's operative parts are encoded and consumed. The same dependency lesson may be retained as a prose reminder, an executable validator, or prose records surfaced through a dense-vector retrieval index; those choices shape execution cost, review evidence, and the invalidation and rollback they admit.

These are coarse families rather than genre taxonomies; when an object bundles several operative parts, the field classifies the relevant part, not the stored object as a whole.

5.1 Prose Artifacts

Prose artifacts include reflections, user facts, project notes, prompts, workflow descriptions, policies, playbooks, and incident reports. Generative-agent memory and verbal-reflection systems retain natural-language observations, reflections, or feedback for later retrieval and use [13,15]. Their strength is flexibility: they are cheap to create, easy to revise, and naturally consumable by language models.

Their limitations follow from the same flexibility: prose is interpreted under underspecified natural-language semantics, so its effect varies with context, prompt position, retrieval order, and model behavior, and it lacks strict artifact-internal mechanisms for scope, binding, precedence, exceptions, and code/data separation, creating prompt-injection risk – LLM-integrated applications "blur the line between data and instructions" [6].

5.2 Symbolic Artifacts

The term symbolic is architectural, not a claim about classical symbolic AI. The distinction is not syntax but operative semantics: a YAML file, Markdown document, or JSON object is symbolic only where a consumer assigns defined consequences to particular fields, values, or structures.

Symbolic artifacts make a different tradeoff from prose: they cost more to specify and cover less by default, but a correct specification can be fast, repeatable, and easy to test, enforce, and roll back. When the specification is wrong, the failure is systematic: a validator may reject legitimate exceptions, or a route table may encode stale tool assumptions.

5.3 Distributed-Parametric Artifacts

Distributed-parametric artifacts include model checkpoints, adapters, embedding spaces, dense-vector indexes, reward models, learned controllers, and other parameterized components encoded across numerical parameters or dense representations [4,19]. The category does not imply identical lifecycle treatment for these artifacts; it marks the shared fact that operative behavior is not localized in directly inspectable symbolic units. They are consumed by numerical procedures such as forward passes, similarity scoring, ranking, and policy selection. They differ from symbolic artifacts even when both are executed by machinery: their behavioral effect is induced by training, optimization, or model-derived dense representation rather than assigned to localized symbolic units.

5.4 Inspection

Form sets the default inspection method: prose calls for reading and prompt review, symbolic artifacts for tests or static checks, and distributed-parametric artifacts for behavioral probes. Direct inspection can weaken in assembled or large-scale settings: a small validator may be easy to test, while a large rule system or retrieval pipeline can make the unit of review the assembly and ranking around individual records rather than any single record [1,16].

6 Lineage and Behavioral Authority

Lineage records an artifact's review-relevant source dependencies and whether it is source material or a derived view, index, or learned update. Lineage here is not a full provenance model; it records the dependency information needed to invalidate, regenerate, or retire retained behavior.

Behavioral authority has three components: the consumer (a model, router, tool runtime, validator, human reviewer, or learning loop), the channel (retrieval, prompt assembly, execution, configuration, ranking, validation, or training), and the force (advice, instruction, enforcement, selection or ranking influence, or learning input). Audit records affect behavior only when a consumer acts on

them; they are not themselves a force. Force ranges from advice, which may be ignored, through instruction and execution-constraining enforcement, to ranking influence and learning input that feeds an automatic training pipeline.

Retained objects often combine operative parts or authority paths. A prompt template may assemble prose instruction with symbolic schema constraints; a semantic-retrieval package may assemble prose records with distributed-parametric embeddings or scoring behavior; and a skill package may assemble prose guidance, code, tests, and activation metadata [17,21]. A Markdown prompt with required sections may be read by an LLM as prompt structure and instruction while validation code checks the same headings as required structure. In such cases, the architectural record should name the relevant operative parts and consumption paths, which carry different affordances and obligations for execution, reuse, review, invalidation, and rollback.

The record exposes repair targets and security failure modes: a stale derived view can govern behavior after its source is fixed, and security failures arise when untrusted prose enters high-authority prompts or opaque retrieval hides poisoned records.

7 Using the Vocabulary: Dependency Example

In practice, the vocabulary becomes a retained-adaptation record. In the example below the first columns record the four fields (substrate and lineage merged) and the last three are review consequences the architect assigns. Each row shows the same dependency lesson retained in a different form; calling all of them "memory" would hide the distinct architectural profiles the fields expose:

Artifact	Substrate / lineage	Form	Authority path / force	Review evidence	Invalidates when	Rollback / delete
Failure reflection	Vector store or file record; derived from run trace.	Prose; dense-vector path if indexed.	LLM via retrieval or prompt context; advice.	Read; probe retrieval.	Correction, deletion, or retirement.	Delete record; refresh index.
Conflict-check script	Repository artifact; source executable.	Symbolic executable.	Tool runtime or validator; enforce-ment.	Unit and integration tests.	Policy or tooling change.	Commit revert or feature flag.
Markdown prompt	Prompt registry; derived from workflow.	Prose instruction.	LLM prompt channel; instruc-tion.	Prompt review.	Source workflow change.	Remove or regenerate entry.
Semantic-retrieval package	Vector store; assembled retrieval package.	Prose records; dense-vector embeddings and scoring path.	Retriever or ranker; ranking influence.	Read records; probe retrieval and ranking.	Poisoning, drift, or retired sources.	Rebuild index or roll back; verify hosted copies removed.

Each row admits a different action: delete and re-index records, feature-flag validators, invalidate derived prompts, and rebuild or roll back retrieval packages.

The record reframes the design question: not whether a lesson is "in memory," but whether its current form and authority are justified – prose advice for the uncertain, symbolic validation or workflow control for the frequent or safety-critical, and retrieval or learned state where generalization outweighs direct inspection.

To apply the record, identify retained artifacts; split bundled objects into operative parts or consumption paths; record substrate, form, lineage, and authority; then assign review evidence, invalidation triggers, and rollback or deletion paths.

8 Applying the Vocabulary to a System Corpus

We applied the record to a corpus of 141 agent-memory and context-engineering systems, each reviewed from source code[1] and classified with the same four fields. The counts below are system-level (presence of an artifact kind, not per-artifact). Within each field we recorded a few practically chosen features, not exhaustive subdivisions: which representational forms are present (Section 5); lineage as authored, imported, or trace-extracted; the consumer, channel, and force of behavioral authority (Section 6); and the activation path, pull or push.

The reproducible core of fifty systems traces to the public comment thread on a widely circulated note on file-based agent "wikis" [10], identified from a frozen snapshot of that thread; the rest of the corpus was gathered opportunistically. Because that seed discussion concerns file- and wiki-style memory, the corpus over-represents it, so we read distributions as descriptive of this collection rather than estimates of the field and rest the argument on properties that do not depend on representativeness.

Within those limits, the corpus supports the paper's central claim. One storage family dominates – 98 of 141 systems keep retained state in plain files or repositories – yet within that single substrate the other fields still pull systems apart:

Among the 98 files/repository systems	Systems
Acts as an enforced gate	53
Includes a distributed-parametric part	37
Pull-only activation	39
Trace-extracted lineage	70

Holding storage fixed, an architect still cannot predict whether retained behavior is enforced or merely advisory, whether it carries learned numerical state, whether it is pulled on demand or pushed unasked, or whether it derives from execution traces. Where retained state lives fixes neither what it is nor how it acts.

[1] Corpus, per-system reviews, the classification contract, and a script that recomputes every count in this section are released as a citable data pack: `https://doi.org/10.5281/zenodo.20763397`.

The same corpus shows why the unit of classification is the operative part, not the system: prose and symbolic parts are each present in all 141 systems, so the form axis discriminates only below the system, at the artifact. The record was nonetheless assignable across the full substrate range, from fine-tuned weights to hosted memory services, so it applies beyond any single mechanism or domain.

9 Limitations and Future Work

The framework is a classification discipline, not a new memory mechanism or full lifecycle theory. The proposed fields, and the features recorded within them, are not exhaustive.

A fuller account should integrate with provenance and dependency-tracking work, and should grade and compose behavioral authority across multi-path artifacts with competing consumers.

The corpus analysis in Section 8 applies the record at scale but is not a controlled validation. Its counts are system-level rather than per-artifact – the vocabulary classifies artifacts and most systems mix several, but we record only whether a system holds at least one of a kind, leaving per-artifact statistics to future work. The corpus is opportunistically assembled rather than a representative sample, and the reproducible core was selected by an AI screen of a comment thread that a second pass showed to be non-exhaustive. The core differs from the rest – distributed-parametric parts appear in 17 of its 50 systems versus 53 of the other 91 – exposing the file-memory bias of the seed. The reviews and their classifications were produced under a saved, fixed contract rather than multi-annotator hand-coding, so reviewer-consistency remains future work. A pre-registered, query-based sampling frame would let any reader reconstruct a larger and less file-biased corpus. Further steps are to apply the record in design reviews and incident analyses, testing whether independent reviewers classify operative parts consistently and whether obligations derived from it expose risks that storage labels alone would miss.

10 Conclusion

Persistent adaptation cannot be classified by storage substrate alone. Storage locates retained state; form, lineage, and authority determine how it acts and what review it admits. The practical question for each retained lesson is whether it should remain quick-to-capture prose, become symbolic control, be surfaced through retrieval, or be learned into numerical state – choices that differ sharply in cost, coverage, behavioral variance, and the review and rollback they admit. Naming those differences lets architects discuss solutions that keep uncertain lessons as prose, move recurring or safety-critical ones toward schemas, validators, tests, or workflow rules, and reserve retrieval or learned state for cases where coverage or generalization matters more than direct inspection. The resulting record gives architectural review of efficiency, security, and sovereignty a precise vocabulary.

References

1. Chen, Z., Xiang, Z., Xiao, C., Song, D., Li, B.: AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. In: Advances in Neural Information Processing Systems (2024), https://openreview.net/forum?id=Y841BRW9rY
2. Couture, S., Toupin, S.: What does the notion of "sovereignty" mean when referring to the digital? New Media & Society **21**(10), 2305–2322 (2019). https://doi.org/10.1177/1461444819865984
3. Dale, R.: Sovereign AI in 2025. Natural Language Processing **31**(5), 1312–1321 (2025). https://doi.org/10.1017/nlp.2025.10007
4. Du, P.: Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers (2026), https://arxiv.org/abs/2603.07670
5. Franch, X., Martínez-Fernández, S., Ayala, C.P., Gómez, C.: Architectural Decisions in AI-Based Systems: An Ontological View. In: Proceedings of the 15th International Conference on the Quality of Information and Communications Technology. pp. 18–27 (2022). https://doi.org/10.1007/978-3-031-14179-9_2
6. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security. pp. 79–90 (2023). https://doi.org/10.1145/3605764.3623985
7. Jia, Z., Li, J., Kang, Y., Wang, Y., Wu, T., Wang, Q., Wang, X., Zhang, S., Shen, J., Li, Q., Qi, S., Liang, Y., He, D., Zheng, Z., Zhu, S.C.: The AI Hippocampus: How Far are We From Human Memory? Transactions on Machine Learning Research (2025), https://openreview.net/forum?id=Sk7pwmLuAY
8. Jiang, P., Lin, J., Shi, Z., Wang, Z., He, L., Wu, Y., Zhong, M., Song, P., Zhang, Q., Wang, H., Xu, X., Xu, H., Han, P., Zhang, D., Sun, J., Yang, C., Qian, K., Wang, T., Hu, C., Li, M., Li, Q., Peng, H., Wang, S., Shang, J., Zhang, C., You, J., Liu, L., Lu, P., Zhang, Y., Ji, H., Choi, Y., Song, D., Sun, J., Han, J.: Adaptation of Agentic AI: A Survey of Post-Training, Memory, and Skills (2025), https://arxiv.org/abs/2512.16301
9. Kang, M., Chen, W.N., Han, D., Inan, H.A., Wutschitz, L., Chen, Y., Sim, R., Rajmohan, S.: ACON: Optimizing Context Compression for Long-horizon LLM Agents (2025), https://arxiv.org/abs/2510.00615
10. Karpathy, A.: On LLM Wikis: File-based Knowledge for Agents. Public gist and comment thread (2026), https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f, accessed 2026-06-18
11. Mei, L., Yao, J., Ge, Y., Wang, Y., Bi, B., Cai, Y., Liu, J., Li, M., Li, Z.Z., Zhang, D., Zhou, C., Mao, J., Xia, T., Guo, J., Liu, S.: A Survey of Context Engineering for Large Language Models (2025), https://arxiv.org/abs/2507.13334
12. Muccini, H., Vaidhyanathan, K.: Software Architecture for ML-Based Systems: What Exists and What Lies Ahead. In: Proceedings of the 1st IEEE/ACM Workshop on AI Engineering–Software Engineering for AI. pp. 121–128 (2021). https://doi.org/10.1109/WAIN52551.2021.00026
13. Park, J.S., O'Brien, J.C., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S.: Generative Agents: Interactive Simulacra of Human Behavior. In: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (2023). https://doi.org/10.1145/3586183.3606763

14. Perry, D.E., Wolf, A.L.: Foundations for the Study of Software Architecture. ACM SIGSOFT Software Engineering Notes **17**(4), 40–52 (1992). https://doi.org/10.1145/141874.141884
15. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language Agents with Verbal Reinforcement Learning. In: Advances in Neural Information Processing Systems 36 (2023), https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html
16. Srivastava, S.S., He, H.: MemoryGraft: Persistent Compromise of LLM Agents via Poisoned Experience Retrieval (2025), https://arxiv.org/abs/2512.16962
17. Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., Anandkumar, A.: Voyager: An Open-Ended Embodied Agent with Large Language Models. Transactions on Machine Learning Research (2024), https://openreview.net/forum?id=ehfRiF0R3a
18. Wang, Z.Z., Mao, J., Fried, D., Neubig, G.: Agent Workflow Memory. In: Proceedings of the 42nd International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 267, pp. 63897–63911. PMLR (2025), https://proceedings.mlr.press/v267/wang25bx.html
19. Yu, Y., Yao, L., Xie, Y., Tan, Q., Feng, J., Li, Y., Wu, L.: Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents (2026), https://arxiv.org/abs/2601.01885
20. Zhang, X., Wang, G., Cui, Y., Qiu, W., Li, Z., Zhu, B., He, P.: Experience Compression Spectrum: Unifying Memory, Skills, and Rules in LLM Agents (2026), https://arxiv.org/abs/2604.15877
21. Zheng, B., Fatemi, M.Y., Jin, X., Wang, Z.Z., Gandhi, A., Song, Y., Gu, Y., Srinivasa, J., Liu, G., Neubig, G., Su, Y.: SkillWeaver: Web Agents Can Self-Improve by Discovering and Honing Skills (2025), https://arxiv.org/abs/2504.07079
22. Zhou, C., Chai, H., Chen, W., Guo, Z., Shan, R., Song, Y., Xu, T., Yang, Y., Yu, A., Zhang, W., Zheng, C., Zhu, J., Zheng, Z., Zhang, Z., Lou, X., Zhang, C., Fu, Z., Wang, J., Liu, W., Lin, J., Zhang, W.: Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering (2026), https://arxiv.org/abs/2604.08224